

MLD — GarageFlow

1. Objectif du MLD

Le Modèle Logique de Données permet de transformer le MCD du projet GarageFlow en structure relationnelle exploitable par une base de données SQL.

Il permet de définir :

- les tables ;
- les clés primaires ;
- les clés étrangères ;
- les relations entre les tables ;
- les contraintes principales ;
- les règles métier liées aux données.

Le choix d'une base relationnelle est cohérent avec le projet, car les données sont fortement liées entre elles : utilisateurs, garages, véhicules, rendez-vous, prestations et interventions.

2. Tables principales

ROLE

ROLE

- id_role PK
- code
- libelle

Rôle : stocker les différents rôles de l'application.

Exemples :

- ADMIN
 - GERANT
 - EMPLOYE
 - CLIENT
-

GARAGE

GARAGE

- id_garage PK
- nom
- adresse

- ville
- code_postal
- telephone
- email
- description
- logo_url
- actif
- date_creation
- date_modification

Rôle : représenter un garage indépendant inscrit sur la plateforme.

UTILISATEUR

UTILISATEUR

- id_utilisateur PK
- id_role FK → ROLE(id_role)
- id_garage FK → GARAGE(id_garage), nullable
- nom
- prenom
- email
- mot_de_passe
- telephone
- actif
- date_creation
- date_modification

Rôle : représenter tous les comptes de la plateforme.

Un utilisateur peut être :

- administrateur ;
- gérant ;
- employé ;
- client.

Remarque :

Le champ `id_garage` est nullable, car un client ou un admin plateforme n'est pas obligatoirement rattaché à un garage.

PLAGE_OUVERTURE

PLAGE_OUVERTURE

- id_plage_ouverture PK
- id_garage FK → GARAGE(id_garage)

- jour_semaine
- heure_debut
- heure_fin
- actif

Rôle : stocker les horaires récurrents d'un garage.

Exemple :

- lundi 08h00-12h00 ;
- lundi 14h00-18h00.

Cette structure permet de gérer les pauses déjeuner.

INDISPONIBILITE

INDISPONIBILITE

- id_indisponibilite PK
- id_garage FK → GARAGE(id_garage)
- id_utilisateur_createur FK → UTILISATEUR(id_utilisateur), nullable
- date_debut
- date_fin
- motif
- date_creation

Rôle : permettre au garage de bloquer un créneau ou une journée de manière exceptionnelle.

Exemples :

- congé ;
 - fermeture exceptionnelle ;
 - absence ;
 - urgence atelier.
-

PRESTATION

PRESTATION

- id_prestation PK
- id_garage FK → GARAGE(id_garage)
- nom
- description
- duree_minutes
- actif
- date_creation
- date_modification

Rôle : représenter les services proposés par un garage.

Exemples :

- vidange ;
- diagnostic ;
- révision ;
- freinage ;
- pneus.

Remarque :

Le prix n'est pas présent dans le MVP, car le projet se concentre sur la prise de rendez-vous, le planning et le suivi d'intervention.

VEHICULE

VEHICULE

- id_vehicule PK
- id_client FK → UTILISATEUR(id_utilisateur)
- marque
- modele
- plaque_immatriculation
- kilometrage
- annee
- carburant
- date_creation
- date_modification

Rôle : stocker les véhicules enregistrés par les clients.

Un client peut posséder plusieurs véhicules. Lors de la prise de rendez-vous, il sélectionne le véhicule concerné.

RENDEZ_VOUS

RENDEZ_VOUS

- id_rendez_vous PK
- id_garage FK → GARAGE(id_garage)
- id_client FK → UTILISATEUR(id_utilisateur)
- id_vehicule FK → VEHICULE(id_vehicule)
- id_prestation FK → PRESTATION(id_prestation)
- date_debut
- date_fin
- statut
- commentaire_client
- date_creation
- date_modification

Rôle : stocker les demandes et réservations de rendez-vous.

Statuts possibles :

- EN_ATTENTE
- CONFIRME
- REFUSE
- ANNULE
- TERMINE

Règle métier :

Un rendez-vous concerne toujours :

- un garage ;
- un client ;
- un véhicule ;
- une prestation.

Dans le MVP, un rendez-vous ne contient qu'une seule prestation.

STATUT_INTERVENTION

STATUT_INTERVENTION

- id_statut_intervention PK
- code
- libelle
- ordre_affichage
- visible_client

Rôle : stocker les statuts possibles d'une intervention.

Exemples :

- VEHICULE_DEPOSE
- DIAGNOSTIC_EN_COURS
- ATTENTE_VALIDATION_CLIENT
- REPARATION_EN_COURS
- VEHICULE_PRET
- VEHICULE_RECUPERE

Le champ `visible_client` permet d'anticiper une évolution future avec des statuts internes non visibles par le client.

INTERVENTION

INTERVENTION

- id_intervention PK
- id_rendez_vous FK → RENDEZ_VOUS(id_rendez_vous), unique
- id_statut_actuel FK → STATUT_INTERVENTION(id_statut_intervention)
- notes_resume
- date_creation
- date_cloture

Rôle : représenter le suivi atelier lié à un rendez-vous confirmé.

Règle métier :

Une intervention est créée lorsqu'un rendez-vous est confirmé.

Un rendez-vous refusé ou annulé ne déclenche pas d'intervention.

Le champ `id_rendez_vous` est unique afin de garantir qu'un rendez-vous ne génère qu'une seule intervention.

HISTORIQUE_STATUT_INTERVENTION

HISTORIQUE_STATUT_INTERVENTION

- id_historique_statut PK
- id_intervention FK → INTERVENTION(id_intervention)
- id_statut_intervention FK → STATUT_INTERVENTION(id_statut_intervention)
- id_utilisateur_modificateur FK → UTILISATEUR(id_utilisateur)
- commentaire
- date_changement

Rôle : conserver l'historique des changements de statut d'une intervention.

Cette table permet de savoir :

- quel statut a été appliqué ;
 - quand il a été appliqué ;
 - par quel utilisateur ;
 - sur quelle intervention.
-

NOTE_INTERNE

NOTE_INTERNE

- id_note_interne PK
- id_intervention FK → INTERVENTION(id_intervention)

- id_auteur FK → UTILISATEUR(id_utilisateur)
- contenu
- date_creation
- date_modification

Rôle : stocker les notes internes du garage liées à une intervention.

Règle métier :

Les notes internes ne sont jamais visibles par le client.

NOTIFICATION

NOTIFICATION

- id_notification PK
- id_destinataire FK → UTILISATEUR(id_utilisateur)
- id_rendez_vous FK → RENDEZ_VOUS(id_rendez_vous), nullable
- id_intervention FK → INTERVENTION(id_intervention), nullable
- type
- canal
- contenu
- lu
- date_creation
- date_lecture

Rôle : stocker les notifications envoyées ou affichées au client.

Types possibles :

- RDV_DEMANDE
- RDV_ACCEPTE
- RDV_REFUSE
- RDV_ANNULE
- STATUT_INTERVENTION_CHANGE
- VEHICULE_PRET

Canaux possibles :

- APP
- EMAIL

Règle métier :

Une notification doit concerner au minimum un contexte métier : un rendez-vous ou une intervention.

LOG_ACTION

LOG_ACTION

- id_log_action PK
- id_utilisateur FK → UTILISATEUR(id_utilisateur), nullable
- id_garage FK → GARAGE(id_garage), nullable
- action
- entite_concernee
- id_entite_concernee
- description
- date_action

Rôle : journaliser les actions importantes réalisées dans l'application.

Exemples :

- création d'un rendez-vous ;
- acceptation d'un rendez-vous ;
- annulation d'un rendez-vous ;
- changement de statut d'intervention ;
- modification des horaires ;
- création d'une prestation.

Cette table améliore la traçabilité et permet de justifier une approche plus professionnelle.

3. Relations principales

ROLE vers UTILISATEUR

Un rôle peut être attribué à plusieurs utilisateurs.

Un utilisateur possède un seul rôle.

Relation :

- ROLE(id_role)
 - UTILISATEUR(id_role)
-

GARAGE vers UTILISATEUR

Un garage peut avoir plusieurs utilisateurs internes.

Un utilisateur peut être rattaché à un garage ou non.

Relation :

- GARAGE(id_garage)
- UTILISATEUR(id_garage)

GARAGE vers PLAGE_OUVERTURE

Un garage peut définir plusieurs plages d'ouverture.
Une plage d'ouverture appartient à un seul garage.

Relation :

- GARAGE(id_garage)
 - PLAGE_OUVERTURE(id_garage)
-

GARAGE vers INDISPONIBILITE

Un garage peut bloquer plusieurs périodes.
Une indisponibilité appartient à un seul garage.

Relation :

- GARAGE(id_garage)
 - INDISPONIBILITE(id_garage)
-

GARAGE vers PRESTATION

Un garage peut proposer plusieurs prestations.
Une prestation appartient à un seul garage.

Relation :

- GARAGE(id_garage)
 - PRESTATION(id_garage)
-

UTILISATEUR vers VEHICULE

Un client peut posséder plusieurs véhicules.
Un véhicule appartient à un seul client.

Relation :

- UTILISATEUR(id_utilisateur)
 - VEHICULE(id_client)
-

UTILISATEUR vers RENDEZ_VOUS

Un client peut prendre plusieurs rendez-vous.
Un rendez-vous est pris par un seul client.

Relation :

- UTILISATEUR(id_utilisateur)
 - RENDEZ_VOUS(id_client)
-

GARAGE vers RENDEZ_VOUS

Un garage peut recevoir plusieurs rendez-vous.
Un rendez-vous concerne un seul garage.

Relation :

- GARAGE(id_garage)
 - RENDEZ_VOUS(id_garage)
-

VEHICULE vers RENDEZ_VOUS

Un véhicule peut être concerné par plusieurs rendez-vous.
Un rendez-vous concerne un seul véhicule.

Relation :

- VEHICULE(id_vehicule)
 - RENDEZ_VOUS(id_vehicule)
-

PRESTATION vers RENDEZ_VOUS

Une prestation peut être choisie dans plusieurs rendez-vous.
Un rendez-vous concerne une seule prestation.

Relation :

- PRESTATION(id_prestation)
 - RENDEZ_VOUS(id_prestation)
-

RENDEZ_VOUS vers INTERVENTION

Un rendez-vous peut déclencher une intervention.
Une intervention est liée à un seul rendez-vous.

Relation :

- RENDEZ_VOUS(id_rendez_vous)
- INTERVENTION(id_rendez_vous)

Contrainte :

`id_rendez_vous` est unique dans la table INTERVENTION.

INTERVENTION vers HISTORIQUE_STATUT_INTERVENTION

Une intervention possède plusieurs lignes d'historique de statut.
Une ligne d'historique appartient à une seule intervention.

Relation :

- INTERVENTION(id_intervention)
 - HISTORIQUE_STATUT_INTERVENTION(id_intervention)
-

STATUT_INTERVENTION vers HISTORIQUE_STATUT_INTERVENTION

Un statut peut être utilisé dans plusieurs historiques.
Une ligne d'historique utilise un seul statut.

Relation :

- STATUT_INTERVENTION(id_statut_intervention)
 - HISTORIQUE_STATUT_INTERVENTION(id_statut_intervention)
-

UTILISATEUR vers HISTORIQUE_STATUT_INTERVENTION

Un utilisateur interne peut modifier plusieurs statuts.
Une modification de statut est réalisée par un seul utilisateur.

Relation :

- UTILISATEUR(id_utilisateur)
 - HISTORIQUE_STATUT_INTERVENTION(id_utilisateur_modificateur)
-

INTERVENTION vers NOTE_INTERNE

Une intervention peut contenir plusieurs notes internes.
Une note interne appartient à une seule intervention.

Relation :

- INTERVENTION(id_intervention)
 - NOTE_INTERNE(id_intervention)
-

UTILISATEUR vers NOTE_INTERNE

Un utilisateur interne peut rédiger plusieurs notes internes.
Une note interne est rédigée par un seul utilisateur.

Relation :

- UTILISATEUR(id_utilisateur)
 - NOTE_INTERNE(id_auteur)
-

UTILISATEUR vers NOTIFICATION

Un utilisateur client peut recevoir plusieurs notifications.
Une notification est destinée à un seul utilisateur.

Relation :

- UTILISATEUR(id_utilisateur)
 - NOTIFICATION(id_destinataire)
-

RENDEZ_VOUS vers NOTIFICATION

Un rendez-vous peut générer plusieurs notifications.
Une notification peut concerner un rendez-vous.

Relation :

- RENDEZ_VOUS(id_rendez_vous)
 - NOTIFICATION(id_rendez_vous)
-

INTERVENTION vers NOTIFICATION

Une intervention peut générer plusieurs notifications.
Une notification peut concerner une intervention.

Relation :

- INTERVENTION(id_intervention)
- NOTIFICATION(id_intervention)

UTILISATEUR vers LOG_ACTION

Un utilisateur peut effectuer plusieurs actions.

Une action peut être réalisée par un utilisateur ou par le système.

Relation :

- UTILISATEUR(id_utilisateur)
- LOG_ACTION(id_utilisateur)

GARAGE vers LOG_ACTION

Un garage peut être concerné par plusieurs logs.

Un log peut concerner un garage.

Relation :

- GARAGE(id_garage)
- LOG_ACTION(id_garage)

4. Contraintes importantes

Unicité des emails

L'email d'un utilisateur doit être unique.

Contrainte :

- UTILISATEUR.email UNIQUE

Unicité véhicule par client

Un même client ne doit pas enregistrer deux fois le même véhicule avec la même plaque.

Contrainte :

- VEHICULE(id_client, plaque_immatriculation) UNIQUE

Pas de double intervention pour un rendez-vous

Un rendez-vous ne peut déclencher qu'une seule intervention.

Contrainte :

- INTERVENTION.id_rendez_vous UNIQUE
-

Cohérence des dates de rendez-vous

La date de début doit être inférieure à la date de fin.

Contrainte :

- RENDEZ_VOUS.date_debut < RENDEZ_VOUS.date_fin
-

Cohérence des horaires d'ouverture

L'heure de début doit être inférieure à l'heure de fin.

Contrainte :

- PLAGE_OUVERTURE.heure_debut < PLAGE_OUVERTURE.heure_fin
-

Cohérence des indisponibilités

La date de début doit être inférieure à la date de fin.

Contrainte :

- INDISPONIBILITE.date_debut < INDISPONIBILITE.date_fin
-

Durée positive pour une prestation

Une prestation doit avoir une durée supérieure à zéro.

Contrainte :

- PRESTATION.duree_minutes > 0
-

Notification liée à un contexte

Une notification doit être liée soit à un rendez-vous, soit à une intervention.

Contrainte :

- NOTIFICATION.id_rendez_vous IS NOT NULL
ou
 - NOTIFICATION.id_intervention IS NOT NULL
-

5. Index recommandés

Pour améliorer les performances, plusieurs index sont prévus.

Index utilisateurs

- UTILISATEUR(id_garage)

Permet de retrouver rapidement les employés d'un garage.

Index prestations

- PRESTATION(id_garage)

Permet de retrouver rapidement les prestations d'un garage.

Index rendez-vous

- RENDEZ_VOUS(id_garage, date_debut)
- RENDEZ_VOUS(id_client)
- RENDEZ_VOUS(id_vehicule)

Permet de charger rapidement :

- le planning d'un garage ;
 - les rendez-vous d'un client ;
 - l'historique d'un véhicule.
-

Index véhicules

- VEHICULE(id_client)

Permet de retrouver rapidement les véhicules d'un client.

Index notifications

- NOTIFICATION(id_destinataire)

Permet de charger rapidement les notifications d'un client.

Index logs

- LOG_ACTION(id_garage)

Permet de filtrer les actions par garage.

6. Justification du MLD

Le MLD respecte la logique métier du projet GarageFlow.

Le garage est au centre de l'organisation, car il possède ses utilisateurs internes, ses horaires, ses prestations, ses indisponibilités et ses rendez-vous.

L'utilisateur est centralisé dans une seule table afin de simplifier l'authentification et la gestion des rôles. Cette approche évite les doublons entre clients, employés, gérants et administrateurs.

Le rendez-vous relie les éléments essentiels du parcours : le client, son véhicule, le garage et la prestation choisie.

L'intervention est séparée du rendez-vous afin de distinguer la réservation de la phase atelier. Cela rend le modèle plus clair et plus évolutif.

L'historique des statuts, les notifications et les logs permettent d'ajouter de la traçabilité, de la sécurité et une meilleure expérience utilisateur.

Ce MLD prépare directement la création de la base de données et le développement du backend.